960121

SYSTEMS THINKING & E-COMMERCE

Session 01

THE ANATOMY OF AN E-COMMERCE

- **The User Interface (Frontend):** The "Storefront" where customers interact with products using HTML, CSS, and JavaScript.
- **The Brain (Backend):** The "Manager" that handles logic, security, and communication via Node.js and Express.
- The **Memory (Database):** The "Warehouse" where all persistent data like product stock and user accounts are stored in SQLite.
- **The Connectors (APIs):** The "Messengers" that allow the frontend to talk to the backend using JSON and HTTP protocols.

THE SOLUTION DEVELOPMENT PROCESS

1. **Requirements & Prerequisites:**
   - Defining the business rules (e.g., "A user must be logged in to checkout").
2. **Logical Design (The Blueprint):**
   - Using Flowcharts or Sequence Diagrams to map how data moves from the "Buy" button to the Database before writing any code.

THE SOLUTION DEVELOPMENT PROCESS

3. **Construction (Implementation):**
   - Building the solution using modular methods, such as Microservices, to ensure the system is scalable.
4. **Validation & Deployment:**
   - Testing if the logic works as intended and deploying the site to a live environment like GitHub Pages.

CORE LOGICAL COMPONENTS

1. **User Identity (Authentication)**
   - **Business Logic:**
     - Verifying who the user is and what they are allowed to do.
   - **Data Flow:**
     - Login Form → Backend Validation → Database Check → Success/Failure Response.

CORE LOGICAL COMPONENTS

2. Product Catalog (Display)

- **Business Logic:**
  - Dynamically fetching and displaying available inventory.
- **Data Flow:**
  - Page Load → API Request → SQL Query → UI Render.

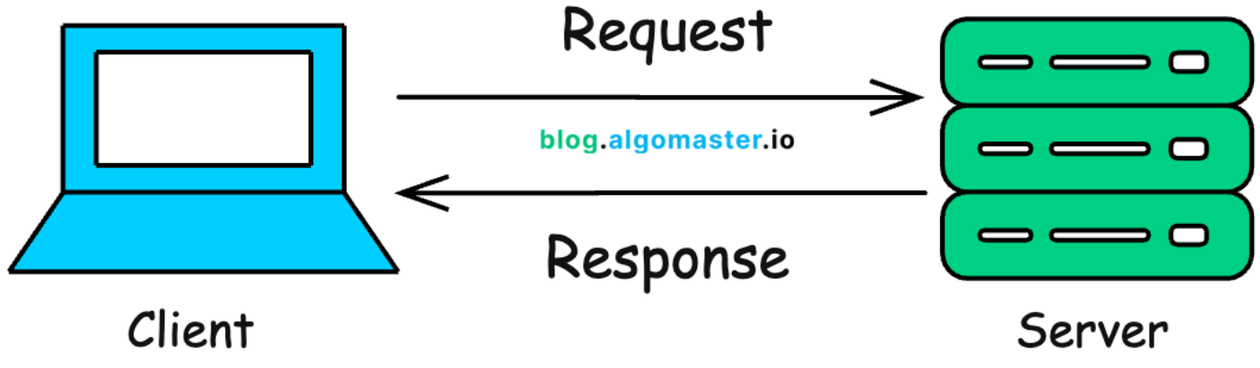
CORE LOGICAL COMPONENTS

3. Transactions (Cart & Checkout)

- Business Logic:
  - Handling the most critical flow—calculating totals, checking stock levels, and saving order history.
- Data Flow:
  - "Add to Cart" Event → State Management → POST Request to Backend → Update SQLite Database.

CLIENT-SERVER ARCHITECTURE

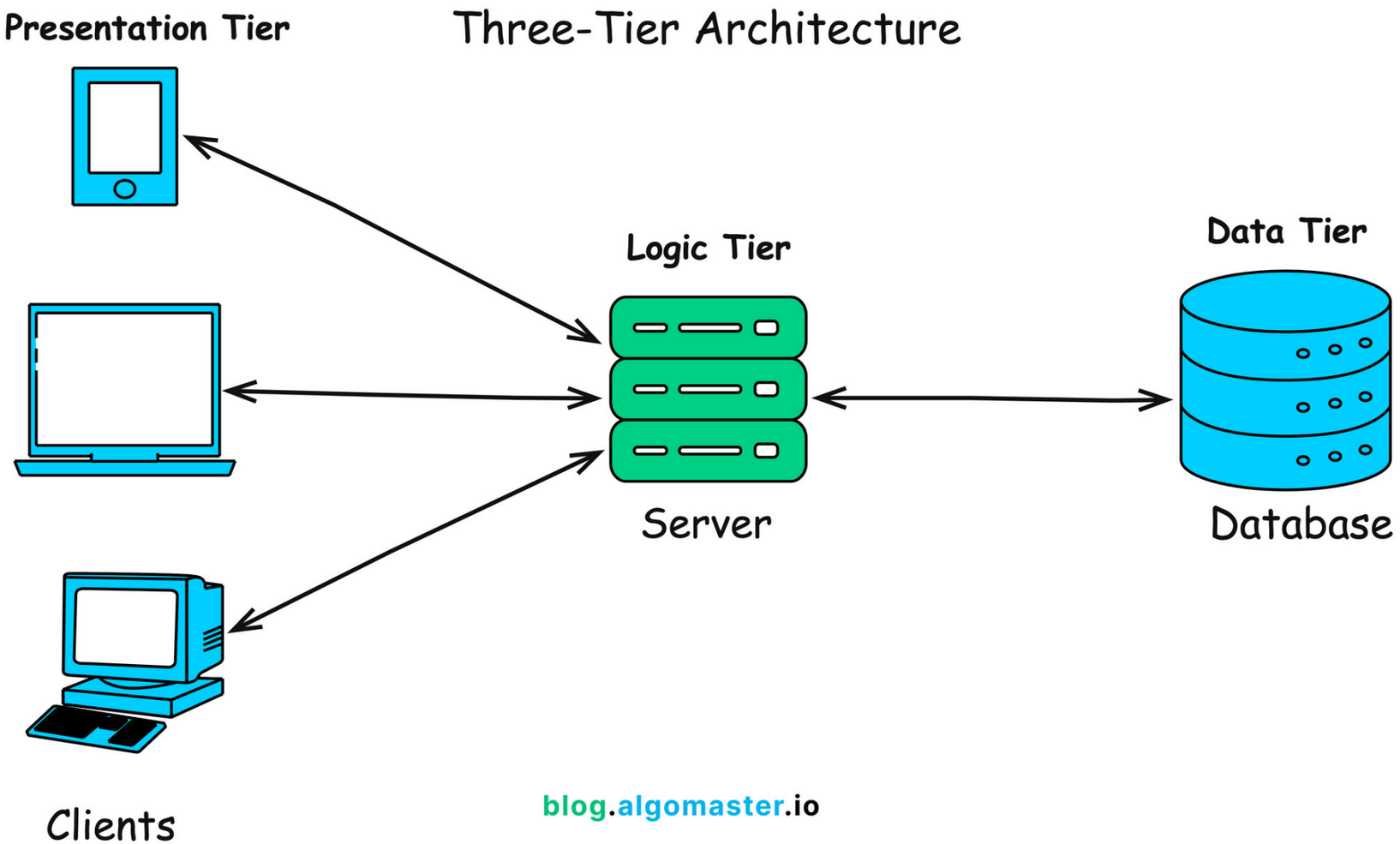
- **The Client (The Requester):** Usually a web browser (Chrome, Safari) or a mobile app that acts as the interface for the user.
- **The Server (The Provider):** A remote computer that waits for requests, processes logic, and sends back the necessary data or files.



CLIENT-SERVER ARCHITECTURE

- **The Request-Response Cycle:**
  - **Request:**
    - The Client asks for a resource (e.g., "Give me index.html" or "Check my login") using **Web Protocols** like HTTP or HTTPS.
  - **Response:**
    - The Server sends back the requested resource or an error message (e.g., "200 OK" or "404 Not Found").

3-TIER ARCHITECTURE



3-TIER ARCHITECTURE

- **Tier 3: The Presentation Tier:**
  - The top-most level that the user sees; it translates tasks and results into something the user can understand (HTML/CSS).
- **Tier 2: The Application Tier (Middle Tier):**
  - The "Engine" of the application; it controls functionality by performing detailed processing (Node.js/Express).
- **Tier 3: The Data Tier (Persistence Tier):**
  - Consists of database servers where information is stored and retrieved (SQLite).

THE 3-LAYER CONCEPT

Layer	Responsibility	Tools/Technologies
Presentation Layer	What the user sees. Handles the layout, design, and user interactions.	HTML, CSS, JavaScript, Bootstrap
Logic Layer	What the app does. Validates data, calculates totals, and manages "Microservice" communication.	Node.js, Express, JavaScript
Persistence Layer	What the app remembers. Safely stores user profiles, product catalogs, and order history.	SQLite, JSON Files

Key Benefit: Separating tiers means you can update the "Look" (Tier 1) without breaking the "Data" (Tier 3).

GitHub

VERSION CONTROL & DEPLOYMENT LOGIC

INTRODUCTION TO GIT (VERSION CONTROL)

- **What is Git?**
  - A distributed version control system that tracks every change made to your code.
- **The "Construction" Logic:**
  - Instead of saving files as final_v1, final_v2, Git creates a logical history of your "Solution construction".
- **Why use Git?**
  - It allows teams to experiment with new business logic without the risk of permanently breaking the main "working" version of the app.

INTRODUCTION TO GIT (VERSION CONTROL)

- **Key Concepts for the Architect:**
  - **Repository (Repo):** The digital folder containing all your project files and their history.
  - **Commit:** A "snapshot" of your code at a specific point in time with a descriptive message of what logic was added.
  - **Push:** Sending your local "Logical Construction" to a remote server (GitHub).

CLASS ACTIVITIES

- (**10 min.**) Search and Download **free** **E-commerce Web Template**
  - Product or Services (Goods or Digital Content)
  - One Page Template
  - Responsive
  - HTML5/CSS3/JS
  - Bootstrap

CLASS ACTIVITIES

- (**10 min.**) **GenAI Integration**: Use AI to explain HTML, CSS, and Bootstrap template structures.
  - Drage HTML file into your choice of GenAI

Prompt:

"*Here is my webpage, explaining HTML , CSS, and Bootstrap structures. Display as a table with one column containing code or a set of code, and one column is their explanations or descriptions*"

CLASS ACTIVITIES

- (**20 min.**) Explore the code structure
  - VS code – localhost (live server)
  - Browser – Page Source/Inspect

CLASS ACTIVITIES

- **(5 min.) Setup GitHub**
  - Create new repository on GitHub (https://github.com/)
  - Link repository with your VS code (local project)
  - Commit and Push
- **(5 min.) Setup GitHub Page**
  - Enable Pages: Configure the repository settings to host the code.
  - your e-commerce storefront will be live at `https://username.github.io/project-name`.